

Answer the Questions

1. In your own words, what is the difference between a delegate and a lambda expression?

- A **delegate** is a type that represents a reference to a method with a specific signature. It can be used to store and pass methods as parameters.
- A **lambda expression** is a short way to write an anonymous function. A lambda can be assigned to a delegate such as `Func` or `Action`.
- In simple terms: **a delegate defines the type of method reference, while a lambda provides the implementation of that method.**

2. Why is `var` still statically typed even though the type isn't written explicitly?

- `var` is statically typed because the compiler determines the variable's type **at compile time** from the value assigned to it.

For example:

```
</> C#  
  
var age = 22;
```

The compiler knows that `age` is an `int`.

So `var` does not mean that the type is unknown or determined at runtime. It simply lets the compiler **infer the type automatically**.

3. Give one real scenario where an anonymous type is more convenient than creating a full class.

- An anonymous type is convenient when you need a temporary object to hold a few related values, especially when processing or selecting data.

For example, if you only need to display an employee's `Name` and `Salary` in a specific operation, you can create:

```
</> C#  
  
var employeeInfo = new  
{  
    Name = "Ali",  
    Salary = 10000  
};
```

Creating a separate `EmployeeInfo` class would be unnecessary if this object is only used temporarily.